ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Capstone project

Building a Movie Data Analysis Pipeline

~ Presented by Group 3 ~

ONE LOVE. ONE FUTURE.

HUST

TABLE OF CONTENTS

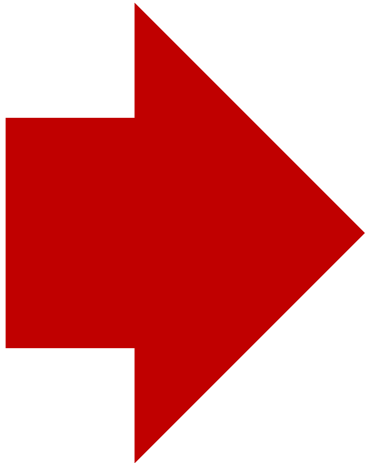
1. Introduction: The Challenge

PR Detection is Too Reactive	• **The Issue:** Negative sentiment explodes on social platforms long before it hits traditional metrics (ratings/box office). • **The Risk:** Without historical comparisons, teams mistake noise for crises (or vice versa) and miss the critical window to protect reputation.
Identifying "True" Virality is Noisy	• **The Issue:** High volume does not always mean virality. Discussion levels vary wildy by genre and budget. • **The Risk:** Marketing teams cannot distinguish organic viral moments from standard traffic, missing the 24–48 hour window for effective amplification.
Recommendations Lack Real-Time Awareness	• **The Issue:** Prioritization systems are often static and fail to account for immediate shifts in audience attention. • **The Risk:** Content surfacing lags behind actual user interest, leading to missed engagement opportunities.

1. Introduction: The Solution

PR Crisis: Statistical Anomaly Detection	• **Approach:** Rather than responding to every small sentiment drop, the system builds a historical baseline to define what “normal” looks like.
Viral Marketing: Relative Velocity Scoring	• **Approach:** Move past raw total volume (which often favors bigger budgets) and instead measure performance vs. expectation.
Content Prioritization: Normalized Trend Ranking	• **Approach:** Add real-time momentum into the recommendation system to prevent feeds from feeling stale.

1. Introduction: The Solution



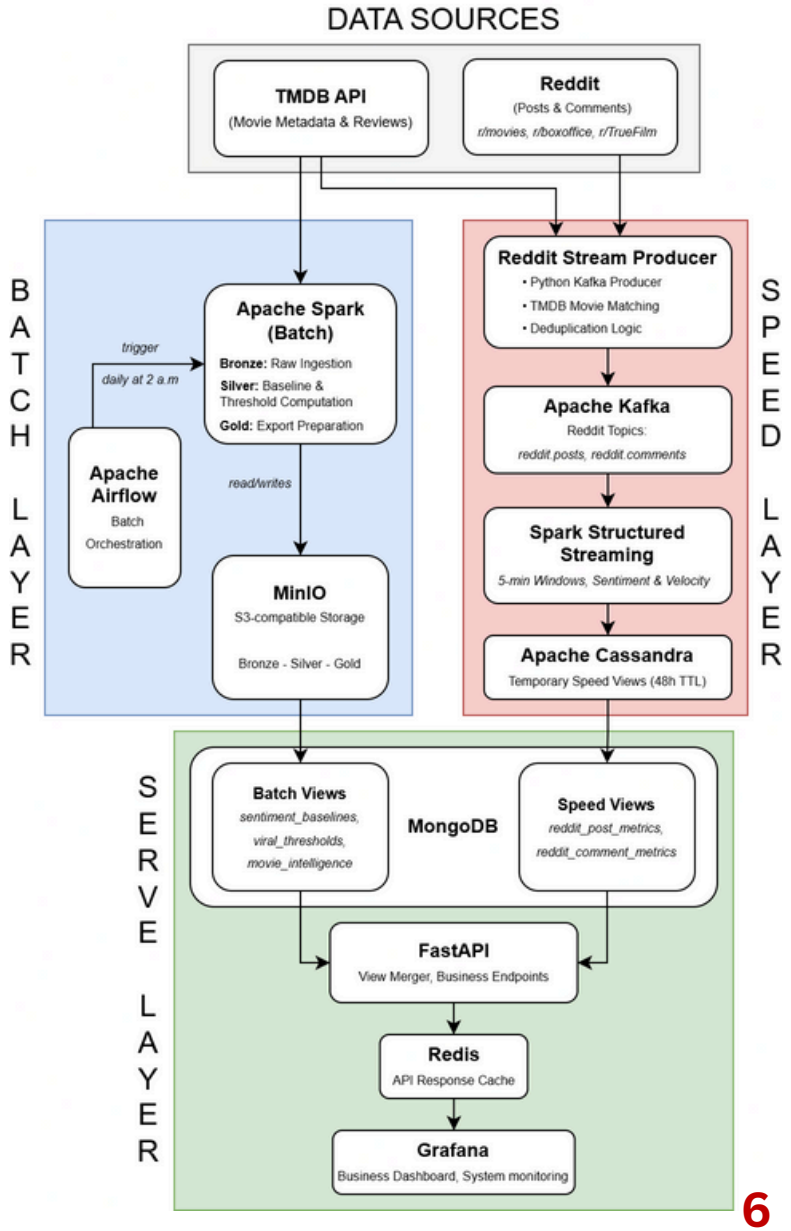
Building an Comprehensive Movie Analysis Pipeline ingesting data from **TMDB (Historical Data Source)** and **Reddit (Real-time Data Source)**

2. Architecture and Design

Lambda Architecture: Dual-path processing for historical accuracy and low-latency freshness

- **Batch Layer →** historical accuracy
- **Speed Layer →** low-latency analytics
- **Serving Layer →** unified query interface

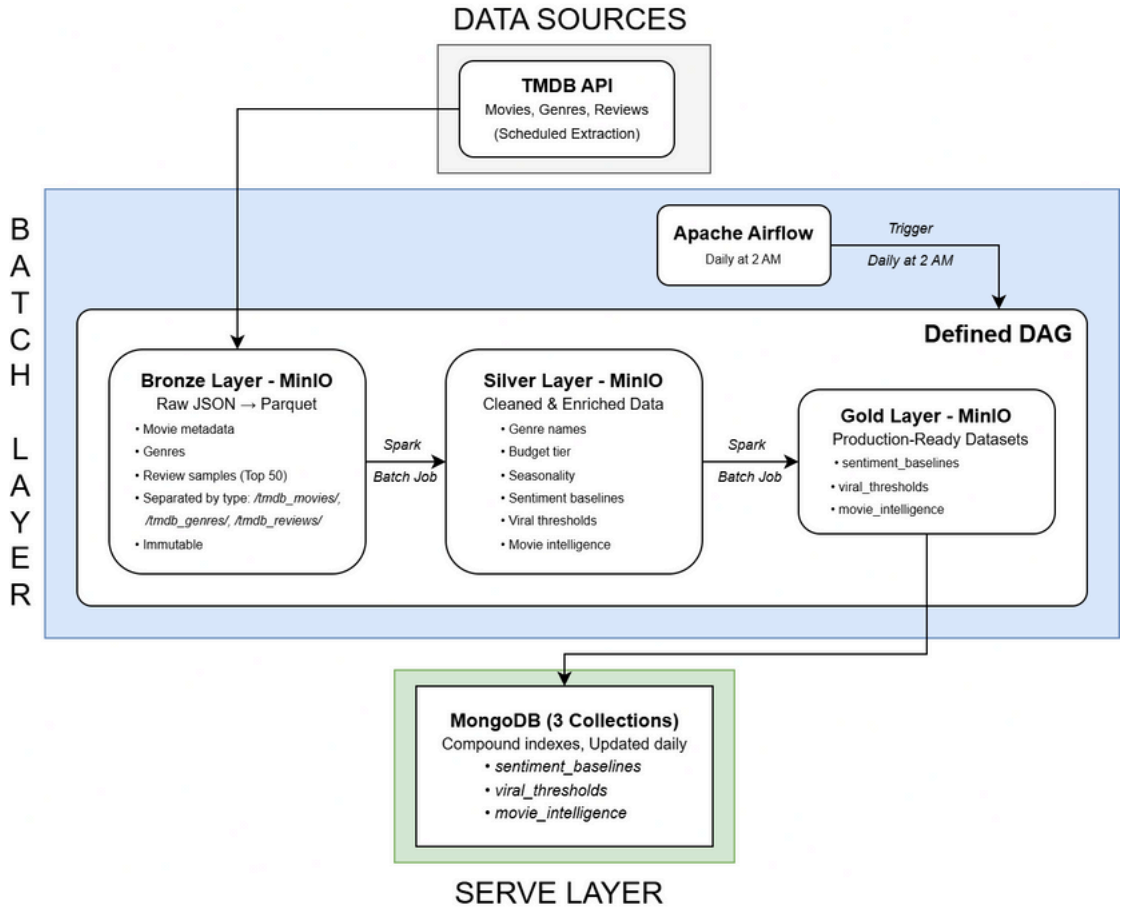
→ Designed to handle high-velocity + large-scale data



2. Architecture and Design

Batch Layer:

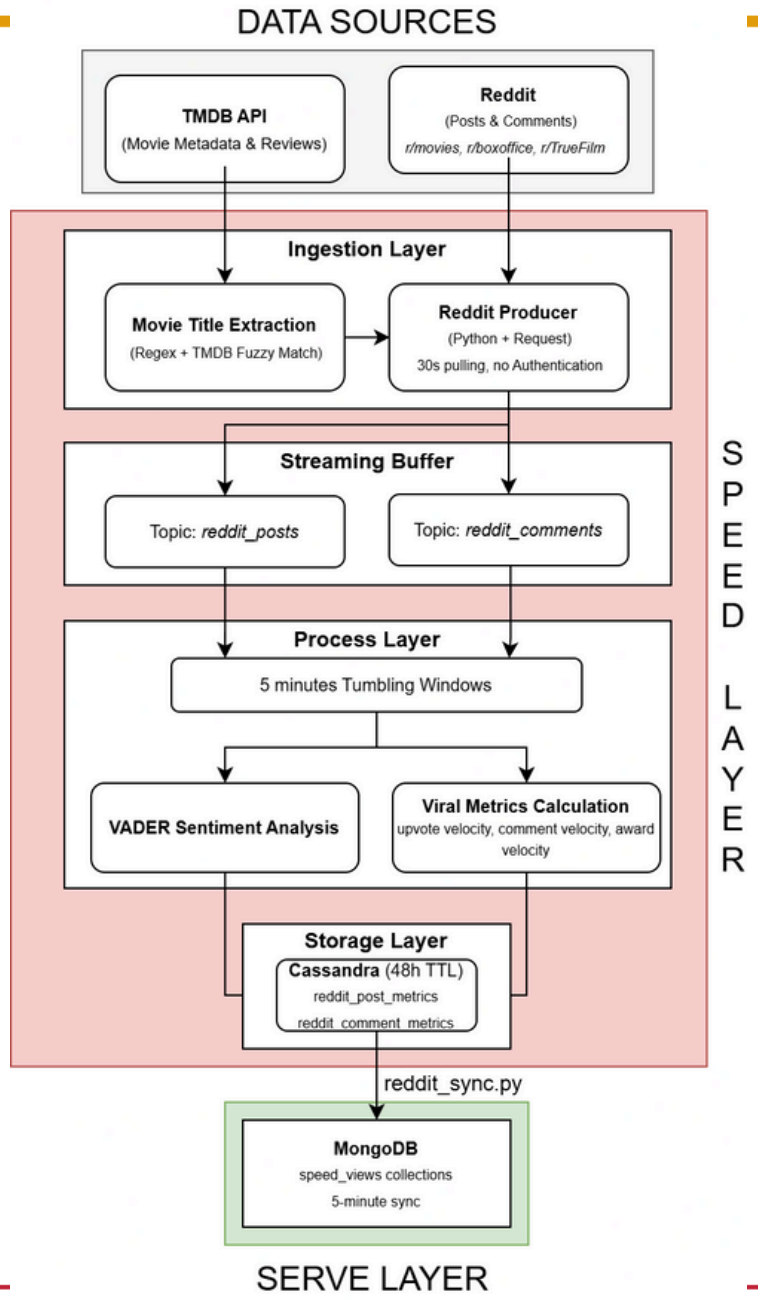
- **Orchestration:** Apache Airflow schedules daily computations at 02:00 UTC.
- **Processing:** Apache Spark for large-scale historical transformations.
- **Storage:** MinIO (S3-compatible) using Bronze (raw), Silver (enriched), and Gold (aggregated) layers.



2. Architecture and Design

Speed Layer (Real-Time):

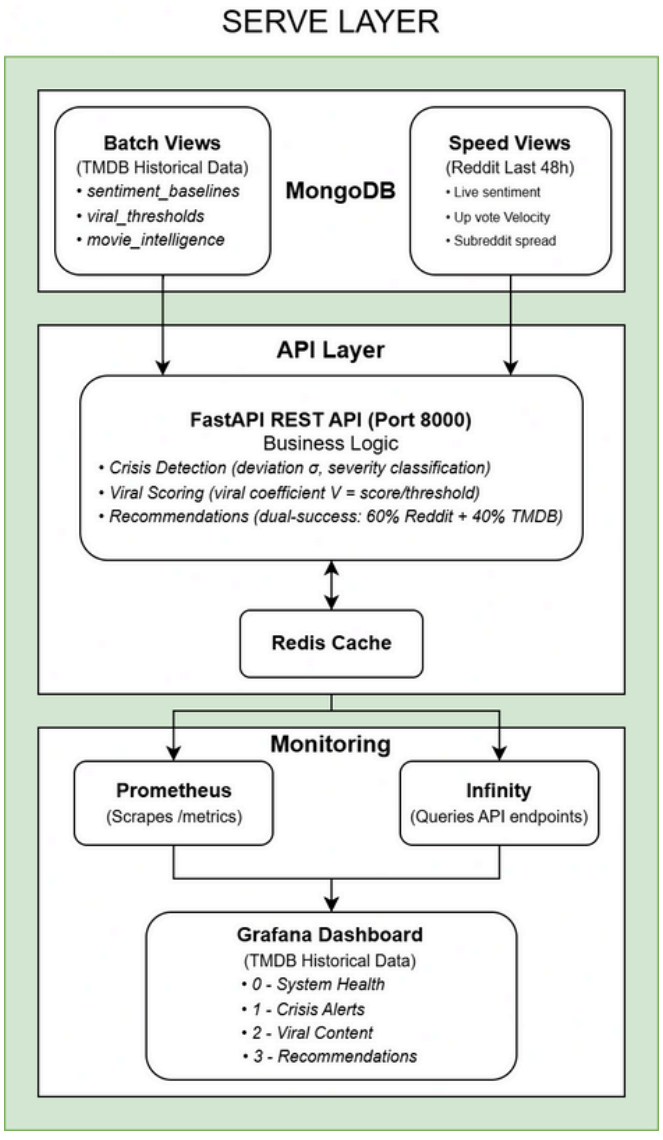
- **Ingestion:** Apache Kafka decouples producers from stream consumers.
- **Processing:** Spark Structured Streaming with 5-minute tumbling windows.
- **Storage:** Apache Cassandra with a 48-hour Time-To-Live (TTL) for automated expiration.



2. Architecture and Design

Serving Layer (Components & Interactions)

- **MongoDB (storage):** serves **6 collections**
  - **Batch (daily):** sentiment_baselines, viral_thresholds, movie_intelligence
  - **Speed:** speed_views (synced from Cassandra, 48h TTL)
- **FastAPI (port 8000):** main query layer + business logic (crisis / viral / recommendations)
- **Redis (cache):** provisioned (30m TTL, LRU eviction) but not actively used (Mongo direct queries + indexes)
- **Prometheus (monitoring):** scrapes metrics of MongoDB and Redis
- **Infinity (Grafana datasource):** calls FastAPI endpoints directly for live JSON panels (real-time state without pre-aggregation)
- **Grafana (visualization)**: dashboards for System Health / Crisis / Viral / Recommendations



2. Architecture and Design

Business Logic Formulas

Goal 1: Crisis Detection (standardized deviation)

- Current sentiment (prefer speed layer if available):

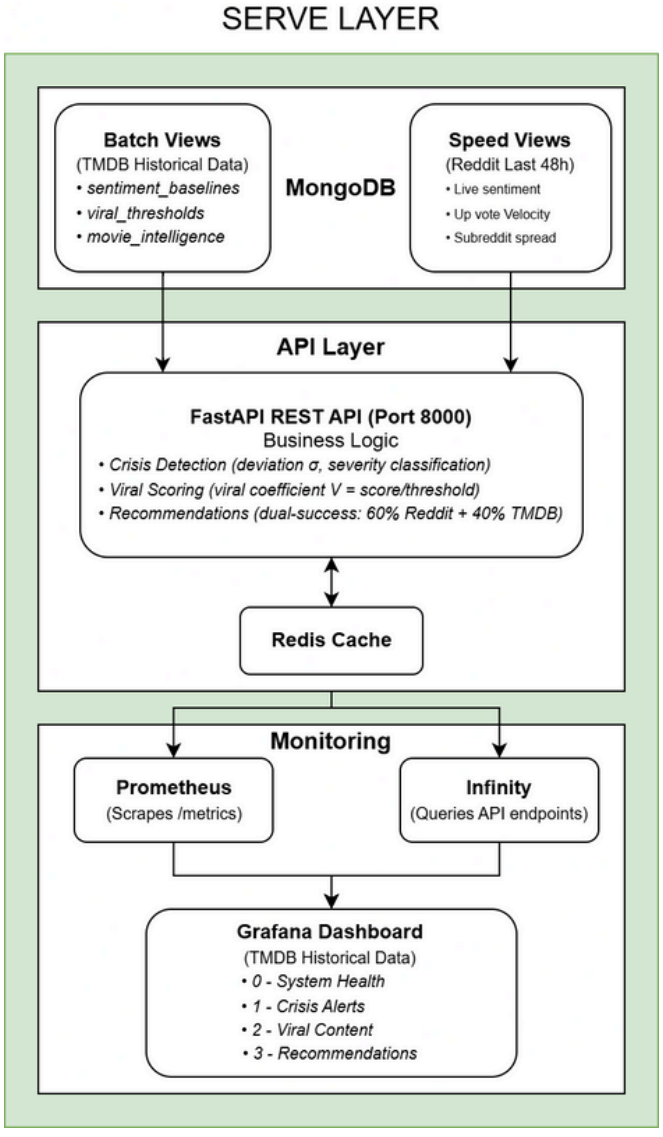
$$S_{current} = \begin{cases} \frac{1}{n}\sum_{i=1}^{n} S_{speed,i} & \text{if speed data exists (48h)} \\ S_{batch} & \text{otherwise} \end{cases}$$

- Standardized deviation vs baseline:

$$\sigma_{deviation} = \frac{S_{current} - \mu_{baseline}}{\sigma_{baseline}}$$

- Severity thresholds:

$$\text{severity} = \begin{cases} \text{critical} & \text{if } \sigma_{deviation} < -70 \\ \text{high} & \text{if } \sigma_{deviation} < -30 \\ \text{warning} & \text{if } \sigma_{deviation} < -25 \\ \text{normal} & \text{otherwise} \end{cases}$$



2. Architecture and Design

Business Logic Formulas

Goal 2: Viral Detection (48h normalized buzz + actions)

- Viral coefficient: $$V = \frac{\sum_{w \in W_{48h}} \text{viral\_score}_w}{\text{avg\_popularity}_{genre}}$$

- Weighted engagement per window:

$$\text{viral\_score} = n_{upvotes} + (2 \times n_{comments}) + (10 \times n_{awards})$$

- Status classification:

$$\text{status} = \begin{cases} \text{viral} & \text{if } V \geq 0.10 \\ \text{trending} & \text{if } 0.05 \leq V < 0.10 \\ \text{growing} & \text{if } 0.01 \leq V < 0.05 \\ \text{stable} & \text{if } V < 0.01 \end{cases}$$

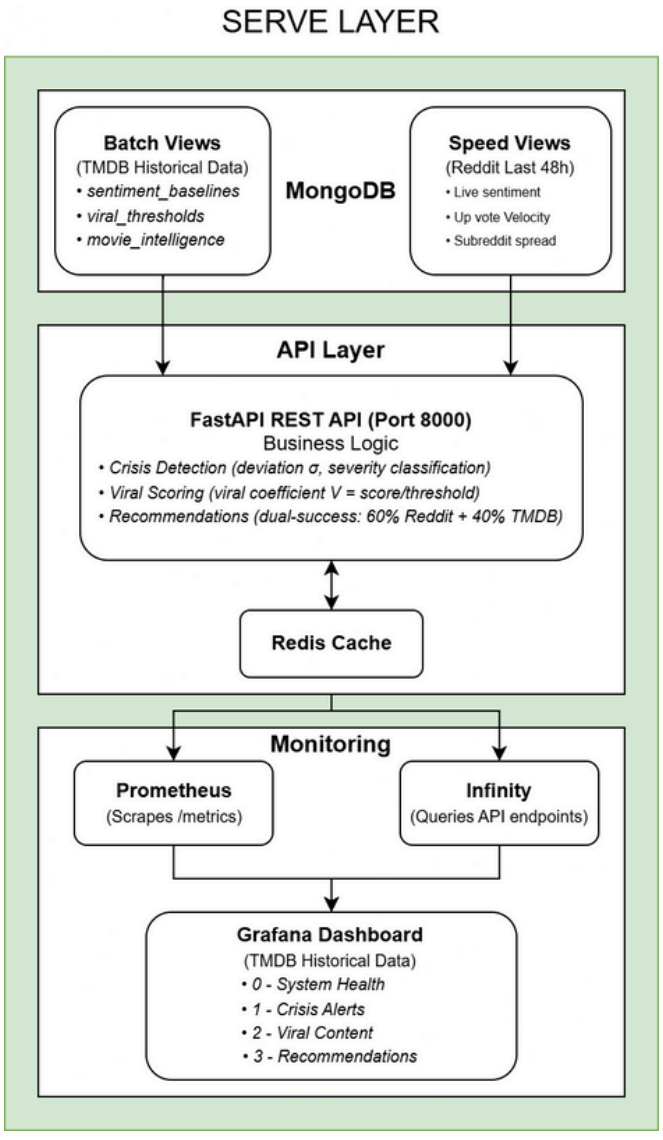
- Marketing opportunity + urgency: $O = V \times (1 + \text{urgency})$

$$\text{urgency} = \text{recency} \times \text{momentum}$$

$$\text{recency} = e^{-\text{age\_hours}/24}, \quad \text{momentum} = \frac{\text{velocity}_{now}}{\text{velocity}_{24h\_ago}}$$

- Recommended actions:

$$\text{action} = \begin{cases} \text{amplify immediately} & \text{if } O \geq 0.10 \text{ and urgency} \geq 0.95 \\ \text{monitor closely} & \text{if } O \geq 0.05 \\ \text{organic growth} & \text{if } O \geq 0.01 \\ \text{evaluate} & \text{if } O < 0.01 \end{cases}$$



2. Architecture and Design

Business Logic Formulas

Goal 3: Recommendations (Reddit + TMDB dual-success)

- Dual-success score (after normalization to 0–100):

$$D_{dual} = 0.6 \times R_{Reddit} + 0.4 \times T_{TMDB}$$

- Min-max normalization:

$$\text{normalized} = \frac{x - x_{min}}{x_{max} - x_{min}} \times 100$$

- Reddit raw score:

$$R_{raw} = W_{engagement} \times \text{decay}_{temporal} \times \text{multiplier}_{volume}$$

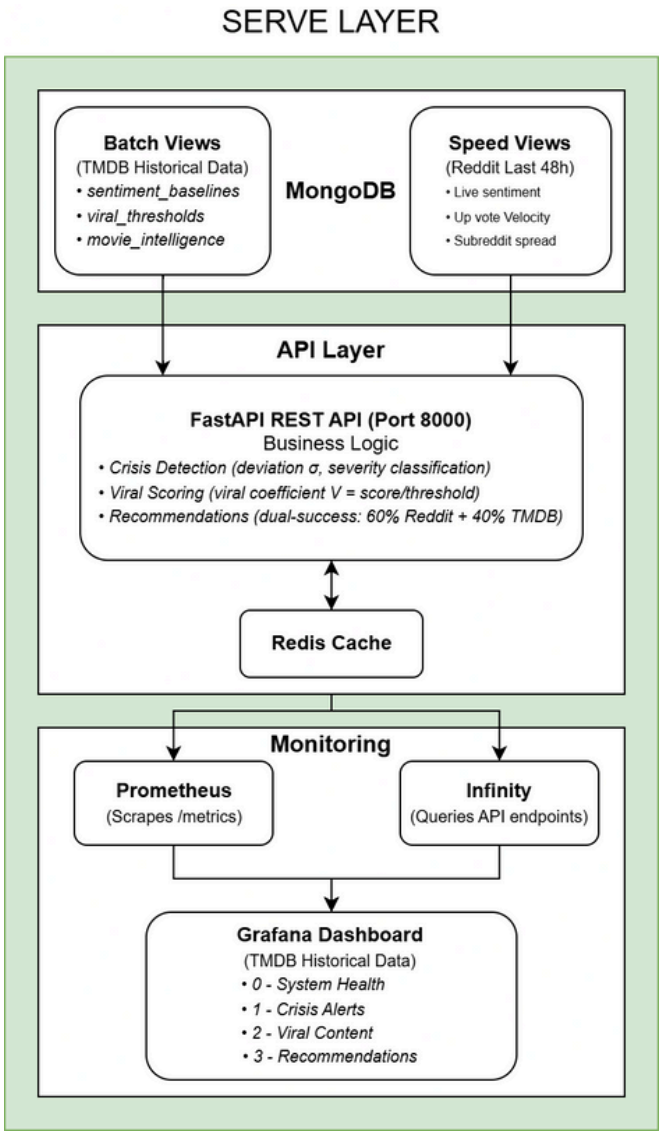
$$W_{engagement} = n_{upvotes} + (2 \times n_{comments}) + (10 \times n_{awards})$$

$$\text{decay}_{temporal} = e^{-\text{age\_hours}/24}$$

$$\text{multiplier}_{volume} = 1 + \frac{\log_{10}(n_{posts} + 1)}{10}$$

- TMDB raw score:

$$T_{raw} = 0.5 \times \text{popularity} + 0.3 \times (\text{vote\_avg} \times 10) + 0.2 \times \log_{10}(\text{votes} + 1)$$



3.1. Source Code Documentation: Batch Layer

- **Orchestration:** The Airflow DAG triggers the pipeline execution
- **Ingestion:**
  - Fetches movie and genre data from TMDB API
  - Persists raw JSON as Parquet files in MinIO
- **Transformation:**
  - Reads Bronze data to compute 3 datasets: Sentiment Baselines, Viral Thresholds, Movie Intelligence
  - Outputs refined Parquet files to MinIO
- **Enrichment:**
  - Appends temporal metadata
  - Finalizes high-quality, refined data for export
- **Export:** Performs a bulk upsert of Gold datasets into MongoDB

```
layers/batch_layer/
├── airflow_dags/
│   └── tmdb_batch_pipeline.py
├── spark_jobs/
│   ├── bronze_ingest.py ........
│   ├── silver_transform.py ....
│   ├── gold_aggregate.py .......
│   ├── export_to_mongo.py .....
│   └── utils/
│       ├── spark_session.py
│       ├── s3_utils.py
│       └── logger.py
└── Dockerfile.airflow
```

3.1. Source Code Documentation: Speed Layer

- **Ingestion:** Scrapes Reddit JSON endpoints
- **Validation:**
  - Cross-references extracted titles against a TMDB cache.
  - Publishes validated JSON messages to Kafka topics
- **Processing:**
  - Applies VADER sentiment analysis via UDF
  - Computes metrics in 5-minute tumbling windows
- **Storage:**
  - 2 tables: reddit_post_metrics, reddit_comment_metrics
  - Implements a 48-hour Time-To-Live (TTL) for temporary storage
- **Synchronization:**
  - Merges data from both tables into MongoDB
  - Syncs to the speed_views collection every 5 minutes.

```
layers/speed_layer/
├── reddit_producers/
│   ├── reddit_stream_producer.py..
│   ├── movie_matcher.py.............
│   └── schemas.py...................
├── streaming_jobs/
│   └── reddit_sentiment_stream.py
├── connectors/
│   └── reddit_sync.py...............
├── cassandra_views/
│   └── reddit_schema.cql ...........
├── kafka_producers/
│   └── setup_topics.py..............
```

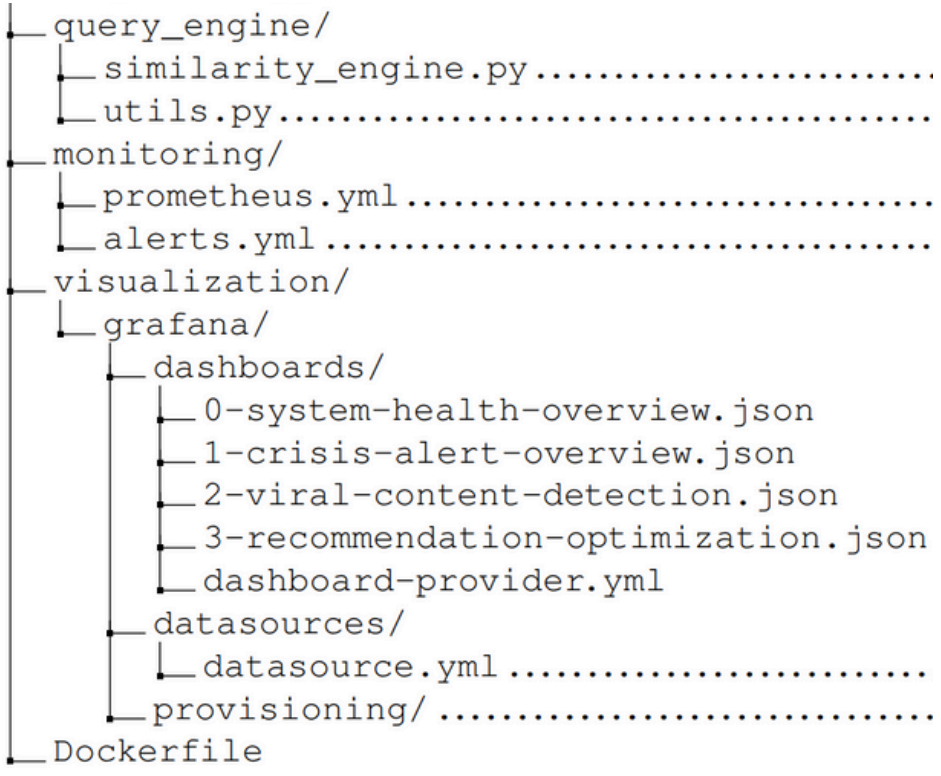
3.1. Source Code Documentation: Serving Layer

- **Request Handling:** FastAPI Router receives HTTP requests and directs them to specific paths
- **Query Routing:** Route handlers analyze the request to select the data source:
  - Real-time queries: Speed Layer
  - Historical analytics: Batch Layer
  - Hybrid queries: Both layers
- **Data Retrieval:** Fetches data using pre-built MongoDB indexes.

```
layers/serving_layer/
├── api/
│   ├── main.py.....................
│   ├── routes/
│   │   ├── health.py..............
│   │   ├── crisis_detection.py.
│   │   ├── viral_detection.py..
│   │   ├── recommendations.py..
│   │   └── utilities.py.........
│   └── schemas/
│       ├── crisis_detection.py
│       ├── recommendations.py..
│       ├── viral_detection.py..
│       └── utilities.py.........
└── mongodb/
    ├── client.py.................
    ├── indexes.py................
    └── queries.py................
```

3.1. Source Code Documentation: Serving Layer

- **Layer Merging:** Normalizes titles using fuzzy matching
- **Core Logic:**
  - Crisis Detection: Calculates deviation sigma against sentiment baselines.
  - Viral Detection: Computes a viral coefficient
  - Recommendations: Generates a dual-success score
- **Validation**: Final results are validated via Pydantic schemas
- **Mornitoring:**
  - Prometheus: tracks latency and error rates
  - Grafana visualizes the output for stakeholders.



3.2. Deployment Strategy

- **Migration approach**
  - Lift-and-shift move from Docker Compose → Kubernetes on GKE, using automation first, then iterative debugging.
- **Cluster target**
  - Small 2-node GKE cluster (4 vCPU, 16GB RAM total) chosen to minimize cost while staying functional.
- **Automated conversion**
  - Used Kompose to convert docker-compose.yml into 60+ Kubernetes YAML manifests (Deployments, Services, PVCs, ConfigMaps).

3.2. Deployment Strategy

Deployment result after debugging

- Achieved 69% success: 18/26 pods running
- Serving layer: 100% (9/9)
- Batch layer: 60% (3/5) (Airflow unstable)
- Speed layer: 50% (6/12) (Kafka/Cassandra infra up, but key streaming services blocked)

Critical blocker

- Missing Reddit API OAuth credentials prevented real-time ingestion/streaming services from operating.
- Without Reddit credentials, real-time features (ingestion + sentiment + full sync) remain disabled → not production-ready.

4. Lessons Learned

Lesson 1: Strategic Planning & Data Suitability

The Challenge:

- **Scope Mismatch:** Initial data sources lacked the high-velocity signals required for a true Big Data streaming pipeline.
- **Impact:** The original system could not satisfy the real-time processing criteria, forcing a project restart.

The Solution:

- Scrapping the initial roadmap to rebuild the plan
- Changed the data sources for the Batch and Speed layers to TMDB and Reddit

Key Takeaway:

- System stability relies on strict pre-deployment configuration checks.

4. Lessons Learned

Lesson 2: API Schema Consistency & Integration

The Challenge:

- **AI Hallucinations:** Relying on AI for code generation led to "hallucinated" endpoints and schema mismatches.
- **Impact:** Broken connections between the Serving Layer (FastAPI) and Grafana (frequent 422 Unprocessable Entity errors).

The Solution:

- **Contract-First Approach:** Defined a strict master list of endpoints and schemas before coding.
- **Single Source of Truth:** Used Pydantic models to enforce strict validation for metrics (e.g., viral_score).

Key Takeaway:

- **Blueprint Required:** AI tools lack system context; a pre-defined API specification is essential.
- **Verification:** Ensure the Data Layer strictly matches the UI/Dashboard expectations.

4. Lessons Learned

Lesson 3: Dependency Validation & Configuration

The Challenge:

- **Fragile Dependencies:** The pipeline relies heavily on external APIs (Reddit/TMDB).
- **Cascading Failures:** Invalid credentials at the ingestion stage caused "silent" failures throughout the entire chain (Ingestion →Spark →Dashboard).

The Solution:

- **"Fail-Fast" Mechanism:** Services verify credentials during the bootstrap phase, not during runtime.
- **Structured Deployment:** Implemented a formal checklist for API quotas and network access.

Key Takeaway:

- **Isolation:** External dependencies are the most fragile points and must be validated first.
- **Stability:** System stability relies on strict pre-deployment configuration checks.

HUST

THANK YOU !

Group members contribution

Name	ID	Work Assignment
Au Trung Phong	20225455	20%: Leader, Refine Serving Layer
Nguyen Nam Khanh	20225448	20%: Build Fundamental Speed Layer
Vu Duc Minh	20225514	20%: Build Fundamental Batch Layer
Tran Sy Minh Quan	20225521	20%: Refine Batch Layer, Speed Layer
Bui Van Huy	20225497	20%: Build Fundamental Serve Layer